

소프트웨어공학

소프트웨어의 구현

- 1 프로그래밍 언어
- 2 구조적 프로그래밍
- 3 검사 기법

01

프로그래밍 언어

구현 단계

설계 단계에서 생성된 설계 명세서를 컴퓨터가 알 수 있는 모습으로 변환하는 과정으로, 프로그래밍 또는 코딩 단계라고도 한다.

- 각 모듈을 특정 프로그래밍 언어를 이용하여 원시 코드로 작성하고 문서화하는 작업이다.
- 구현은 설계를 철저히 반영시키고, 원시코드를 간단명료하게 작성하며, 디버깅 및 변경, 검사가 용이하도록 해야 한다.
- 구현 단계에서는 사용할 프로그래밍 언어와 코딩 스타일 등을 결정해야 한다.

01

프로그래밍 언어

- 프로그래밍 언어

프로그래밍 언어	컴퓨터시스템을 작동시키는 소프트웨어를 작성하기 위한 언어
----------	---------------------------------

저급언어	고급언어
하드웨어에 관련된 직접 제어가 가능한 언어	하드웨어 관련 지식 없어도 프로그램 작성이 가능한 언어

- 프로그래밍 언어의 분류

제1세대 언어(저급언어)	기계어, 어셈블리 언어
제2세대 언어	FORTTRAN, COBOL, ALGOL60, BASIC
제3세대 언어	범용 : PL/1, Pascal, Modula-2, C, Ada, C++, SIMULA, Smalltalk 트수 : CHILL, RPG, Lisp, Prolog, APL
제4세대 언어(4GL)	Non-procedural language MANTIS, IDEAL, RAMIS II, SQL

01

프로그래밍 언어

- 프로그래밍 언어의 선정 기준
 - 구현 단계에서 사용할 프로그래밍 언어를 선정하기 위한 기준은 다음과 같다.
 - 친밀감
 - 언어의 능력
 - 처리의 효율성
 - 과거의 개발 실적
 - 알고리즘과 계산상의 난이도
 - 자료 구조의 난이도
 - 성능 고려 사항들
 - 프로그램 구조
 - 프로그램의 길이
 - 이식성
 - 대상 업무의 성격
 - 소프트웨어의 수행 환경
 - 개발 담당자의 경험과 지식
 - 컴파일러의 이용 가능성

01

프로그래밍 언어

- 코딩 스타일

코딩의 표준화	프로그래머마다 코딩하는 방법이 다르므로 이러한 코딩 방법의 일관성을 유지하고 좋은 코딩을 위해 제시된 표준을 코딩 스타일(coding style)이라고 한다.
---------	--

01

프로그래밍 언어

- 코딩 스타일은 다음과 같다.
 - 프로그램 논리를 명확하게 작성해야 한다.
 - 지나치게 기교를 부리지 않는다.
 - 수식은 간결하고 직접적으로 표현한다.
 - 임시 변수의 사용은 금한다.
 - 혼동을 초래하는 변수명은 사용하지 말고, 일관성 있는 변수명을 사용한다.
 - 블록 구조를 사용한다.
 - 들여쓰기(Indentation)를 사용한다.
 - 판단에 맞는 정확한 제어 구조를 사용한다.

01

프로그래밍 언어

- 코딩 스타일은 다음과 같다.
 - IF 뒤에 바로 IF가 오지 않도록 하고, Else 뒤에 Null이 되지 않게 한다.
 - 모듈화와 서브루틴을 사용한다.
 - 매개 변수가 5개 이상인 모듈은 유의한다.
 - 하나의 변수를 다목적으로 사용하지 않는다.
 - 너무 깊은 중첩 구조는 피한다.

01

프로그래밍 언어

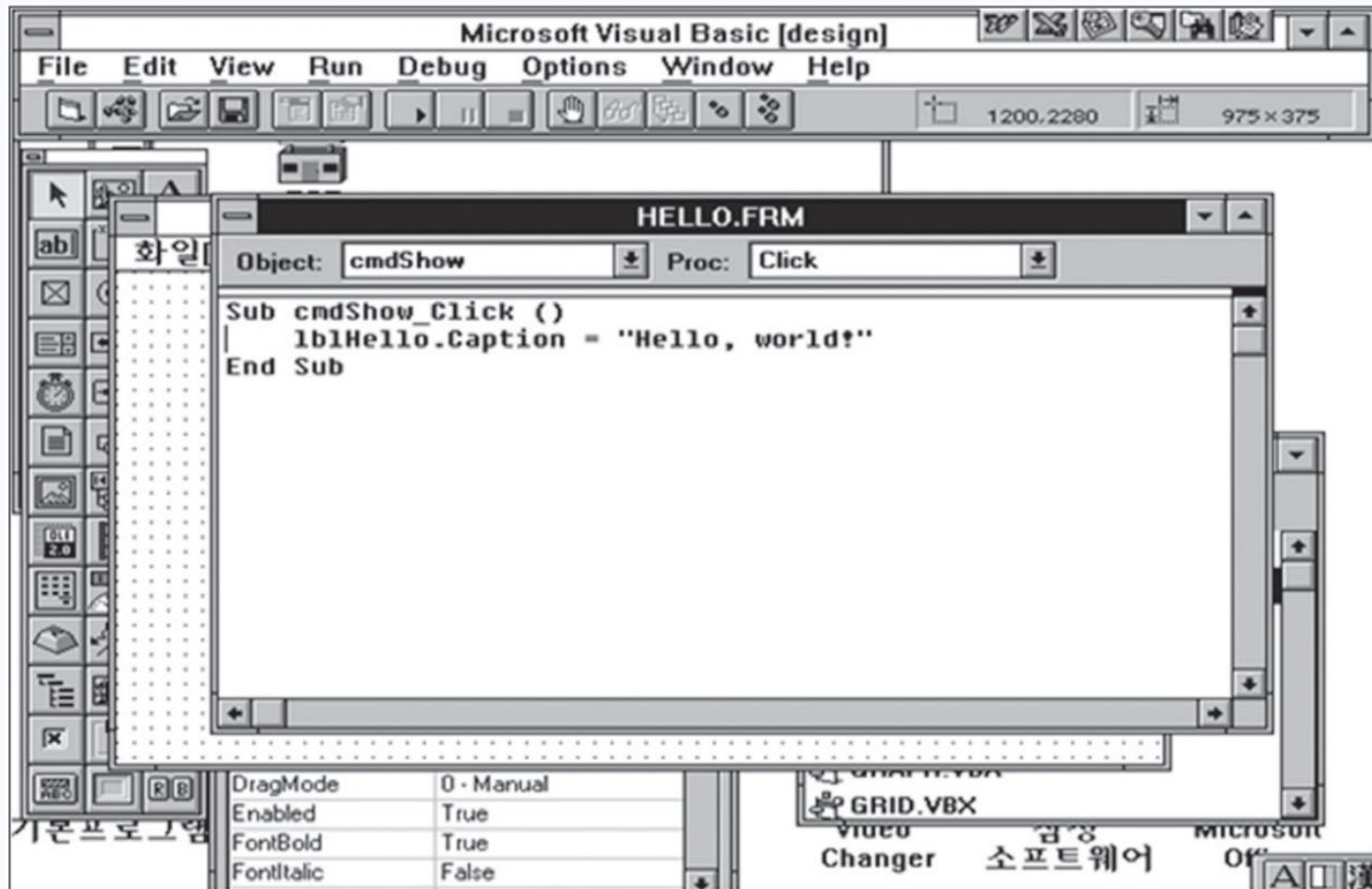
- Visual Basic의 경우
 - ① 원하는 윈도우를 그린다.
 - ② 버튼, 텍스트 박스의 속성을(properties)를 설정



01

프로그래밍 언어

- Visual Basic의 경우
 - ③ 연계된 사건에 대한 코드 작성



구조적 프로그래밍

- 구조적 프로그래밍

구조적 프로그래밍	신뢰성 있는 소프트웨어의 생산과 코딩의 표준화 등을 위해 개발된 방법이다.
-----------	---

- 구조적 프로그래밍의 기본 제어

순차(sequence)	명령을 순서적으로 나열한다.
선택(selection)	특정 논리에 기초하여 명령을 선택한다.
반복(iteration)	순환을 제공한다.

- 순차, 선택, 반복을 사용하면 프로그래밍의 복잡도를 줄여주고, 유지보수가 용이하다.
- 분기(GOTO) 없이 프로그래밍 하여 읽고, 테스트하기 쉽다.
- 오류 없는 프로그램 구성으로 품질을 향상시킨다.

02

구조적 프로그래밍

- 프로그래밍 규칙
 - 프로그램의 제어 흐름을 선형화한다.
 - 단일 입구와 단일 출구만 가지게 한다.
 - GOTO문을 사용하지 않는다.
 - 순차, 선택, 반복의 세 가지 기본 제어 구조를 사용한다.

검사(test)

소프트웨어 품질보증 활동의 하나로, 소프트웨어에 대한 요구사항의 만족도 및 예상 결과에 실제결과의 차이점을 여러 방법을 사용하여 검사하고 평가하는 일련의 과정을 의미한다.

- 소프트웨어 품질을 평가하는 작업이며 분석이나 설계, 코딩 결과를 최종적으로 점검하는 과정이다.
- 오류를 발견하기 위해 프로그램이나 시스템을 수행하는 과정이다.
- 검사의 목적은 소프트웨어를 구성하는 요소들이 잘 조화를 이루며 정상적으로 동작하고 성능이 요구에 맞다는 것을 보장하기 위해서이다.

03 검사 기법

- 성공적인 검사는 아직 발견되지 않은 오류를 찾아내는 것이다.

검사 사례 (test case)	검사 자료나 실행될 조건을 의미하는 것으로, 소프트웨어 오류를 찾아낼 수 있는 가능성이 가장 높은 테스트를 얻어내는 것이다.
----------------------	---

- 검사 기법

검사 기법	최소한의 시간과 노력으로 대부분의 오류를 찾아내기 위해 검사 사례를 설계하는 방법이다.
-------	--

- 검사 사례를 설계하는 검사 기법에는 화이트 박스 테스트와 블랙 박스 테스트가 있다.
- 검사 사례 설계의 고려사항
 - 모듈 내의 모든 독립적인 경로가 적어도 한 번은 수행되어야 한다.
 - 가능한 한 복잡한 논리는 배제시킨다.
 - 임의의 조건을 만족시켜야 한다.
 - 내부 자료 구조를 사용하여 테스트를 수행한다.

03

검사 기법

- 검사 기법

- 화이트 박스 테스트

화이트 박스 테스트 (white box test)	모듈의 원시 코드를 오픈시킨 상태에서 원시 코드의 논리적인 모든 경로를 검사하여 검사 사례를 설계하는 방법이다.
-----------------------------------	--

- 설계된 절차에 초점을 둔 구조적 테스트로 프로시저(절차) 설계의 제어 구조를 사용하여 검사 사례를 설계하며, 테스트 과정의 초기에 적용된다.
 - 모듈 안의 작동을 직접 관찰한다.
 - 원시 코드(모듈)의 모든 문장을 한 번 이상 수행함으로써 수행된다.
 - 프로그램의 제어 구조에 따라 선택, 반복 등의 분기점 부분을 수행함으로써 논리적 경로를 제어한다.

03

검사 기법

- 검사 기법

- 화이트 박스 테스트

화이트 박스 테스트 (white box test)	모듈의 원시 코드를 오픈시킨 상태에서 원시 코드의 논리적인 모든 경로를 검사하여 검사 사례를 설계하는 방법이다.
-----------------------------------	---

- 각 조건에서의 참과 거짓의 모든 논리적 결정이 적어도 한 번 이상 실행된다.
 - 화이트 박스 테스트 기법에는 기초 경로 검사, 제어 구조 검사
(조건 검사, 루프 검사, 데이터 흐름 검사) 등이 있다.

- 검사 기법
 - 화이트 박스 테스트
 - 기초 경로 검사
 - ✓ 기초 경로 검사(basic path testing)는 Tom McCabe가 제안한 것으로 대표적인 화이트 박스 테스트 기법이다.
 - ✓ 검사 사례 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주고, 이 측정 결과는 실행 경로의 기초를 정의하는 데 지침으로 사용된다.
 - 기초 경로 검사
 - ① 설계나 원시 코드를 기초로 해서 흐름도를 작성한다.
 - ② 흐름도의 논리적 복잡도를 측정한다.
 - ③ 독립 경로들의 기초 집합을 결정한다.
 - ④ 기초 집합의 각 경로를 실행시키는 검사 사례를 선정한다.

03

검사 기법

- 검사 기법
 - 화이트 박스 테스트
 - 제어 흐름도

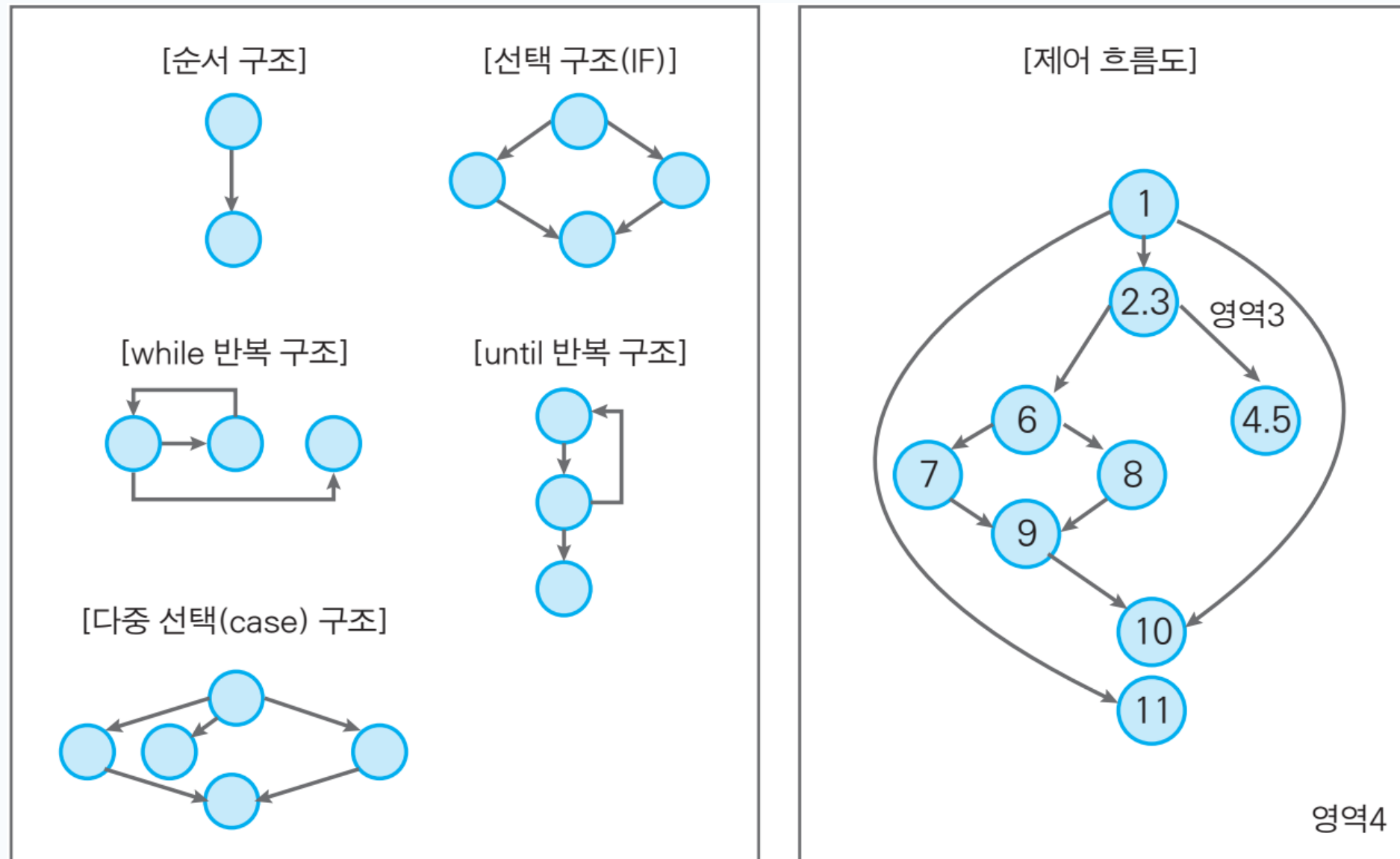
제어 흐름도	제어 흐름도를 표현하기 위해 사용되는 그래프로 프로그램 그래프, 흐름 그래프라고도 한다.
--------	---

✓ 제어 흐름도는 각 제어마다 다음과 같은 기호를 사용하여 표시한다.

노드(원)	절차적 명령문을 나타낸다.
화살표	제어의 흐름도를 나타낸다.
영역	화살표와 노드(원)로 둘러싸인 구역으로, 외부 구역도 하나의 영역으로 포함한다.

03 검사 기법

- 검사 기법
 - 화이트 박스 테스트
 - 제어 흐름도



03

검사 기법

- 검사 기법
 - 화이트 박스 테스트
 - 순환 복잡도

순환 복잡도

한 프로그램의 논리적인 복잡도를 측정하기 위한 소프트웨어의 척도로, 제어 흐름도 이론에 기초를 둔다.

- ✓ 순환 복잡도를 이용하여 계산된 값은 프로그램의 독립적인 경로의 수를 정의하고, 모든 경로가 한 번 이상 수행되었음을 보장하기 위해 행해지는 테스트 횟수의 상한선을 제공 한다.
- ✓ 제어 흐름도 G에서 순환 복잡도 $V(G)$ 는 다음과 같은 방법으로 계산할 수 있다.
 - ◆ 순환 복잡도는 제어 흐름도의 영역 수와 일치하므로 영역 수를 계산한다.
 - ◆ $V(G) = E - N + 2 \rightarrow E$ 는 화살표 수, N 은 노드의 수

03

검사 기법

- 검사 기법
 - 화이트 박스 테스트
 - 제어 구조 검사

조건 검사 (condition testing)	프로그램 모듈 내에 있는 논리적 조건을 검사하는 검사 사례 설계 기법이다. 프로그램에 있는 각 조건을 검사하는 데 초점을 맞추는 방법으로 조건 검사 범위의 측정이 간단하다.
루프 검사 (loop testing)	프로그램의 반복(loop) 구조에 초점을 맞춰 실시하는 검사 사례 설계 기법이다. 루프 검사는 단순루프, 중첩 루프, 연결 루프, 비구조적 루프의 네 가지를 사용하여 정의할 수 있다. 초기화 오류, 인덱싱 및 증가 오류, 경계값 오류 등을 발견할 수 있다.
데이터 흐름 검사 (data flow testing)	프로그램에서 변수의 정의와 변수 사용의 위치에 초점을 맞춰 실시하는 검사 사례 설계 기법이다

03

검사 기법

- 검사 기법

- 블랙 박스 테스트

블랙 박스 테스트 (black box test)	소프트웨어가 수행할 특정 기능을 알기 위해서 각 기능이 완전히 작동되는 것을 입증하는 검사로서 기능 검사라고도 한다.
-------------------------------	---

- 부정확하거나 누락된 기능, 인터페이스 오류, 자료 구조나 외부 데이터베이스 접근에 따른 오류, 행위나 성능 오류, 초기화와 종료 오류 등을 발견하기 위해 사용되며 테스트 과정의 후반부에 적용된다.
- 소프트웨어 산물의 각 기능별로 적절한 정보 영역(입·출력)을 정하여 적합한 입력에 대한 출력의 정확성을 점검한다.
- 블랙 박스 테스트의 종류에는 동치 분할 검사, 경계값 분석, 원인-효과 그래프 검사, 오류 예측 검사, 비교 검사 등이 있다

- 검사 기법

- 블랙 박스 테스트

- 동치 분할 검사(Equivalence Partitioning Testing)

- ✓ 동치 분할 검사는 입력 자료에 초점을 맞춰 검사 사례를 만들고 검사하는 방법으로 동 등 분할 기법이라고도 한다.
 - ✓ 프로그램의 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 하여 검사 사례를 정하고, 해당 입력 자료에 맞는 결과가 출력되는지 확인하는 방법 이다.

- 경계값 분석(Boundary Value Analysis)

- ✓ 경계값 분석은 입력 자료에만 치중한 동치 분할 기법을 보완하기 위한 기법이다.
 - ✓ 입력 조건의 중간값보다 경계값에서 오류가 발생할 확률이 높다는 점을 이용하여 입력 조건의 경계값을 검사 사례로 선정하여 검사한다.

- 검사 기법
 - 블랙 박스 테스트
 - 원인-효과 그래프 검사(Cause-Effect Graphing Testing)
 - ✓ 원인-효과 그래프 검사는 입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계 적으로 분석하여 효용성 높은 검사 사례를 선정하여 검사하는 기법이다.
 - ✓ 프로그램의 외부 명세에 의한 입력 조건과 입력으로부터 발생하는 출력을 논리적으로 연결시킨 그래프로 표현하여 검사한다.
 - 오류 예측 검사(Fault Based Testing) = Mutation Testing
 - ✓ 오류 예측 검사는 과거의 경험이나 확인자의 감각으로 검사하는 기법이다.
 - ✓ 다른 블랙 박스 테스트 기법으로는 찾아낼 수 없는 오류를 찾아내는 일련의 보충적 검사 기법이며, 데이터 확인 검사라고도 한다.

- 검사 기법
 - 블랙 박스 테스트
 - 비교 검사(Comparison Testing)
 - ✓ 여러 버전의 프로그램에 동일한 검사 자료를 제공하여 동일한 결과가 출력되는지 검사하는 기법이다.